



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра суперкомпьютеров и квантовой информатики

Отчет по заданию практикума Системы Линденмайера

Группа: 523

Повжик Юрий Максимович
Почта: s02210514@gse.cs.msu.ru

Москва, 2025

Содержание

1	Программный код.....	3
2	Графики	9

1 Программный код

```
// Test selection via environment variable LSYS_TEST (integer 1..5). Default = 1.
//
// Usage examples:
//   mpirun -np 4 ./lsystem 20000 20           # runs default test
// (LSYS_TEST unset => test 1)
//   LSYS_TEST=3 mpirun -np 8 ./lsystem 20000 20   # run test 3
//   export LSYS_TEST=4
//   mpirun -np 16 ./lsystem 20000 20           # persistent env var
//
// test mapping (built-in):
//   1 - D0L L1: a->b, b->ab, axiom=a   (deterministic)
//   2 - D0L L2: a->ab, b->bc, axiom=a   (deterministic)
//   3 - graph L1: a->ab, b->bc, axiom=a (deterministic for graphs)
//   4 - graph L2: a->aa [0.001], axiom=a (stochastic)
//   5 - graph L3: a->ab [0.01], b->a [0.01], axiom=a (stochastic)

#include <mpi.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <map>
#include <random>
#include <sstream>
#include <algorithm>
#include <cassert>
#include <cstdlib>

using namespace std;

struct StochasticRule { vector<string> productions; vector<double> probabilities; };
using ProdList = vector<pair<string,double>>;
map<char, ProdList> RULES;
string AXIOM = "a";
const long long SAFETY_THRESHOLD = 50'000'000LL;

void load_builtin_test_case(int test_case) {
    RULES.clear();
    AXIOM = "a";
    if (test_case == 1) {
        RULES['a'].push_back({"b", 1.0});
        RULES['b'].push_back({"ab", 1.0});
    } else if (test_case == 2) {
        RULES['a'].push_back({"ab", 1.0});
        RULES['b'].push_back({"bc", 1.0});
    } else if (test_case == 3) {
        RULES['a'].push_back({"ab", 1.0});
```

```

        RULES['b'].push_back({"bc",1.0});
    } else if (test_case == 4) {
        RULES['a'].push_back({"aa",0.001});
    } else if (test_case == 5) {
        RULES['a'].push_back({"ab",0.01});
        RULES['b'].push_back({"a",0.01});
    } else {
        RULES['a'].push_back({"b",1.0});
        RULES['b'].push_back({"ab",1.0});
    }
    for (auto &kv : RULES) {
        double s=0; for(auto &pr:kv.second) s+=pr.second;
        if (s>1.0 + 1e-12) for(auto &pr:kv.second) pr.second /= s;
    }
}

string produce_symbol(char c, mt19937 &rng, uniform_real_distribution<double>
&ud) {
    auto it = RULES.find(c);
    if (it == RULES.end()) return string(1,c);
    const ProdList &pl = it->second;
    if (pl.size()==1 && fabs(pl[0].second - 1.0) < 1e-12) return pl[0].first;
    double r = ud(rng);
    double acc = 0.0;
    for (const auto &pr : pl) {
        acc += pr.second;
        if (r < acc) return pr.first;
    }
    return string(1,c);
}

string update_data_stochastic(const string &data, mt19937 &rng,
uniform_real_distribution<double> &ud) {
    string out; out.reserve(data.size()*2 + 16);
    for (char c : data) out += produce_symbol(c, rng, ud);
    return out;
}

void mpi_send_string(const string &s, int to, int tag) {
    int n = (int)s.size();
    MPI_Send(&n,1,MPI_INT,to,tag,MPI_COMM_WORLD);
    if (n>0) MPI_Send(s.data(), n, MPI_CHAR, to, tag+1000, MPI_COMM_WORLD);
}

string mpi_recv_string(int from, int tag) {
    MPI_Status st; int n=0; MPI_Recv(&n,1,MPI_INT,from,tag,MPI_COMM_WORLD,&st);
    string s; if (n>0) { s.resize(n); MPI_Recv(&s[0], n, MPI_CHAR, from,
tag+1000, MPI_COMM_WORLD, &st); }
    return s;
}

```

```

void local_neighbor_balance(string &local, int rank, int size) {
    for (int phase=0; phase<2; ++phase) {
        int partner = -1;
        if ((rank % 2) == phase) partner = rank + 1;
        else partner = rank - 1;
        if (partner < 0 || partner >= size) {
            MPI_Barrier(MPI_COMM_WORLD);
            continue;
        }
        int mylen = (int)local.size();
        int theirlen = 0;
        MPI_Sendrecv(&mylen,1,MPI_INT,partner,100+phase,
                    &theirlen,1,MPI_INT,partner,100+phase,
                    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        int total = mylen + theirlen;
        int left = min(rank, partner), right = max(rank, partner);
        int desired_left = total / 2;
        if (rank == left) {
            if (mylen > desired_left) {
                int excess = mylen - desired_left;
                string tosend = local.substr(mylen - excess);
                local.erase(mylen - excess);
                int send_size = (int)tosend.size();
                MPI_Send(&send_size,1,MPI_INT,partner,200+phase,MPI_COMM_WORLD);
                if (send_size>0) MPI_Send(tosend.data(), send_size, MPI_CHAR,
partner, 300+phase, MPI_COMM_WORLD);
            } else if (mylen < desired_left) {
                int recv_size=0;
                MPI_Recv(&recv_size,1,MPI_INT,partner,200+phase,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
                if (recv_size>0) {
                    string rec; rec.resize(recv_size);
                    MPI_Recv(&rec[0], recv_size, MPI_CHAR, partner, 300+phase,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                    local += rec;
                }
            }
        } else {
            int desired_right = total - desired_left;
            if (mylen > desired_right) {
                int excess = mylen - desired_right;
                string tosend = local.substr(0, excess);
                local.erase(0, excess);
                int send_size = (int)tosend.size();
                MPI_Send(&send_size,1,MPI_INT,partner,200+phase,MPI_COMM_WORLD);
                if (send_size>0) MPI_Send(tosend.data(), send_size, MPI_CHAR,
partner, 300+phase, MPI_COMM_WORLD);
            } else if (mylen < desired_right) {
                int recv_size=0;

```

```

        MPI_Recv(&recv_size,1,MPI_INT,partner,200+phase,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        if (recv_size>0) {
            string rec; rec.resize(recv_size);
            MPI_Recv(&rec[0], recv_size, MPI_CHAR, partner, 300+phase, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            local = rec + local;
        }
    }
    MPI_Barrier(MPI_COMM_WORLD);
}

void run_parallel(int m, int k, int test_case, int rank, int size) {
    load_builtin_test_case(test_case);
    random_device rd; mt19937 rng((unsigned int)(rd() + rank*7919 + 12345));
    uniform_real_distribution<double> ud(0.0,1.0);
    string local = (rank==0) ? AXIOM : string();
    ofstream statf;
    if (rank==0) { statf.open("stat.txt"); statf << "# t"; for (int r=0;r<size;++r) statf << " p" << r; statf << "\n"; statf.flush(); }
    vector<long long> all_lengths(size,0);

    for (int t=1; t<=m; ++t) {
        local = update_data_stochastic(local, rng, ud);
        if ((t % k) == 0) {
            long long local_len = (long long)local.size();
            MPI_Gather(&local_len,1,MPI_LONG_LONG,
all_lengths.data(),1,MPI_LONG_LONG, 0, MPI_COMM_WORLD);
            if (rank==0) {
                long long total=0; for(int i=0;i<size;++i) total +=
all_lengths[i];
                statf << t;
                if (total==0) for(int i=0;i<size;++i) statf << " " << 0.0;
                else for(int i=0;i<size;++i) statf << " " <<
((double)all_lengths[i] / (double)total);
                statf << "\n"; statf.flush();
            }
            local_neighbor_balance(local, rank, size);
        }
    }

    long long mylen_ll = (long long)local.size();
    long long total_len = 0;
    MPI_Allreduce(&mylen_ll, &total_len, 1, MPI_LONG_LONG, MPI_SUM, MPI_COMM_WORLD);

    if (rank==0) {
        if (total_len > SAFETY_THRESHOLD) {

```

```

        vector<long long> final_lengths(size,0);
        MPI_Gather(&mylen_ll,1,MPI_LONG_LONG,
final_lengths.data(),1,MPI_LONG_LONG, 0, MPI_COMM_WORLD);
        ofstream fout("output.txt");
        fout << "# total_length = " << total_len << "\n";
        fout << "# per_process_lengths:\n";
        for (int i=0;i<size;++i) fout << "proc" << i << " " <<
final_lengths[i] << "\n";
        fout.close();
        if (statf.is_open()) statf.close();
        cout << "Total length " << total_len << " exceeds safety threshold;
summary written to output.txt\n";
    } else {
        vector<int> recv_counts(size,0), displs(size,0);
        int local_int = (int)mylen_ll;
        MPI_Gather(&local_int,1,MPI_INT, recv_counts.data(),1,MPI_INT, 0,
MPI_COMM_WORLD);
        int sum=0; for (int i=0;i<size;++i) { displs[i]=sum; sum +=
recv_counts[i]; }
        vector<char> global(sum);
        MPI_Gatherv(local.c_str(), local_int, MPI_CHAR, global.data(),
recv_counts.data(), displs.data(), MPI_CHAR, 0, MPI_COMM_WORLD);
        ofstream fout("output.txt");
        fout << string(global.begin(), global.end()) << "\n";
        fout.close();
        if (statf.is_open()) statf.close();
        cout << "Final total length: " << total_len << " (output.txt
written)\n";
    }
    } else {
        int local_int = (int)mylen_ll;
        MPI_Gather(&local_int,1,MPI_INT, nullptr,0,MPI_INT, 0, MPI_COMM_WORLD);
        MPI_Gatherv(local.c_str(), local_int, MPI_CHAR, nullptr, nullptr,
nullptr, MPI_CHAR, 0, MPI_COMM_WORLD);
    }
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank=0, size=1;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc != 3) {
        if (rank==0) {
            cerr << "Usage: " << argv[0] << " <m> <k>\n";
            cerr << "Program reads test id from environment variable LSYS_TEST
(1..5). Default LSYS_TEST=1\n";
            cerr << "Example: LSYS_TEST=3 mpirun -np 4 ./lsystem 20000 20\n";
        }
    }
}

```

```

        MPI_Finalize();
        return 1;
    }

    int m = atoi(argv[1]);
    int k = atoi(argv[2]);
    if (m <= 0 || k <= 0) {
        if (rank==0) cerr << "m and k must be positive integers\n";
        MPI_Finalize();
        return 1;
    }

    int test_case = 1;
    const char* env = getenv("LSYS_TEST");
    if (env != nullptr) {
        int v = atoi(env);
        if (v >= 1 && v <= 5) test_case = v;
        else {
            if (rank==0) cerr << "Warning: LSYS_TEST out of range; using default
1\n";
        }
    }

    if (rank==0) {
        cout << "Running L-system test " << test_case << " with m=" << m << " k="
<< k << " on " << size << " ranks\n";
    }

    run_parallel(m,k,test_case,rank,size);

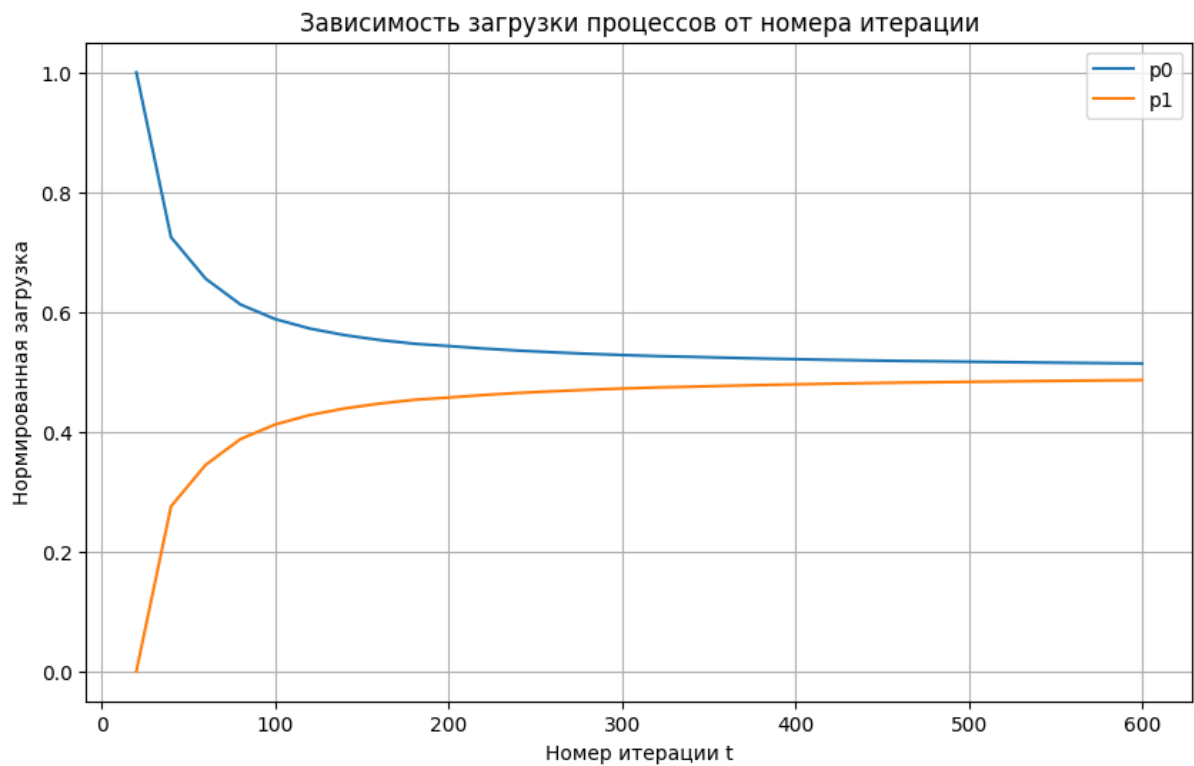
    MPI_Finalize();
    return 0;
}

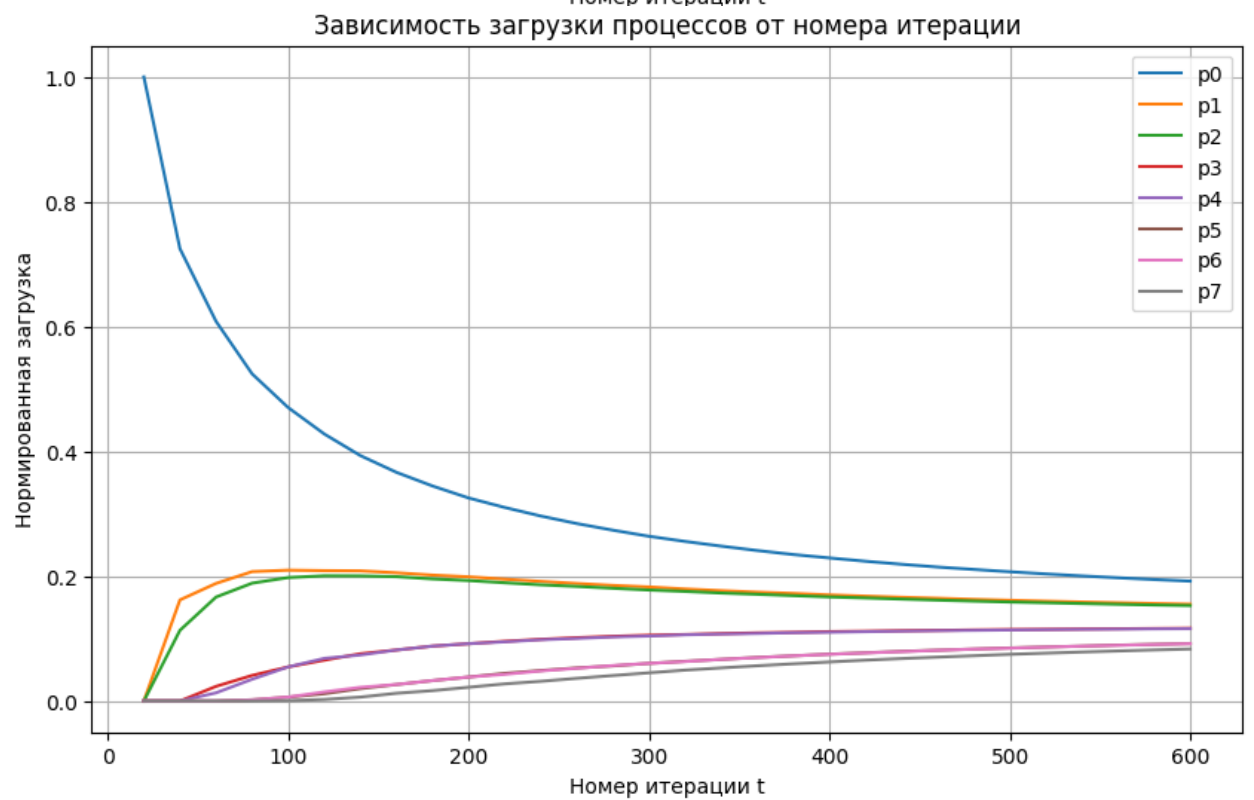
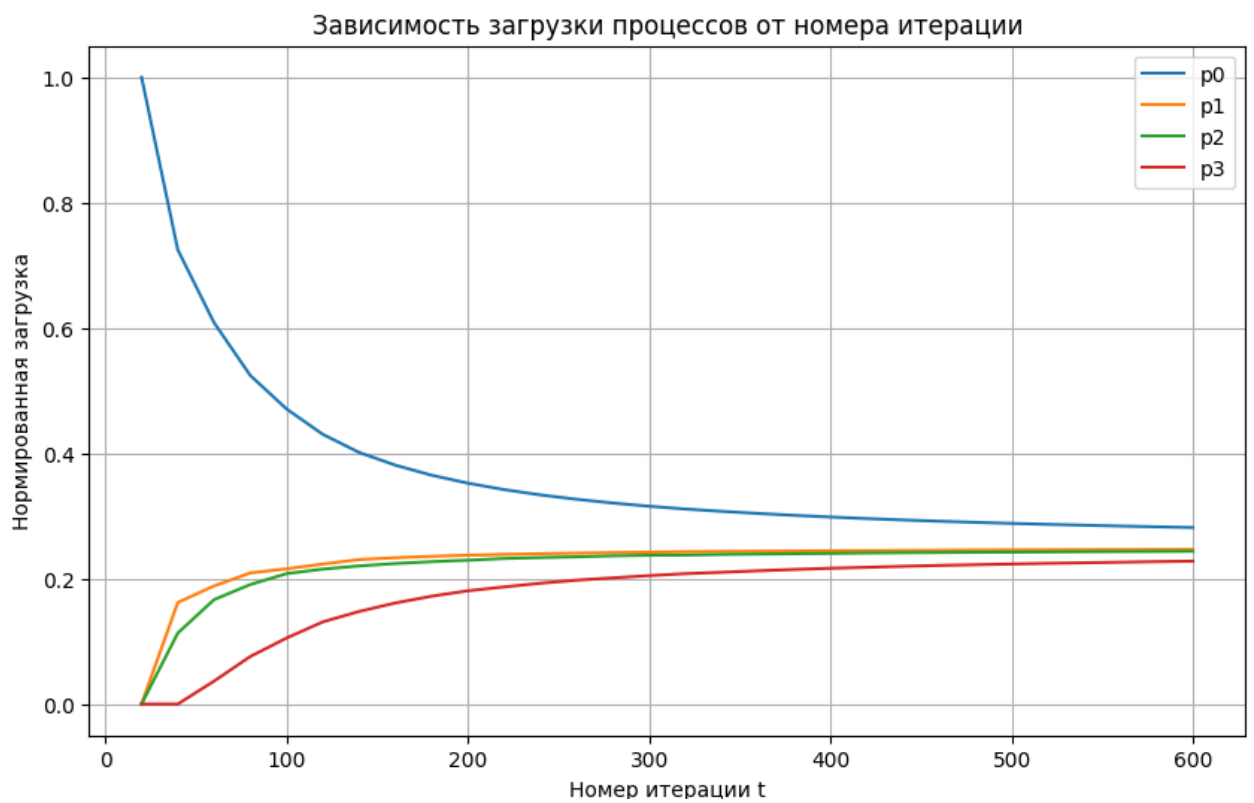
```

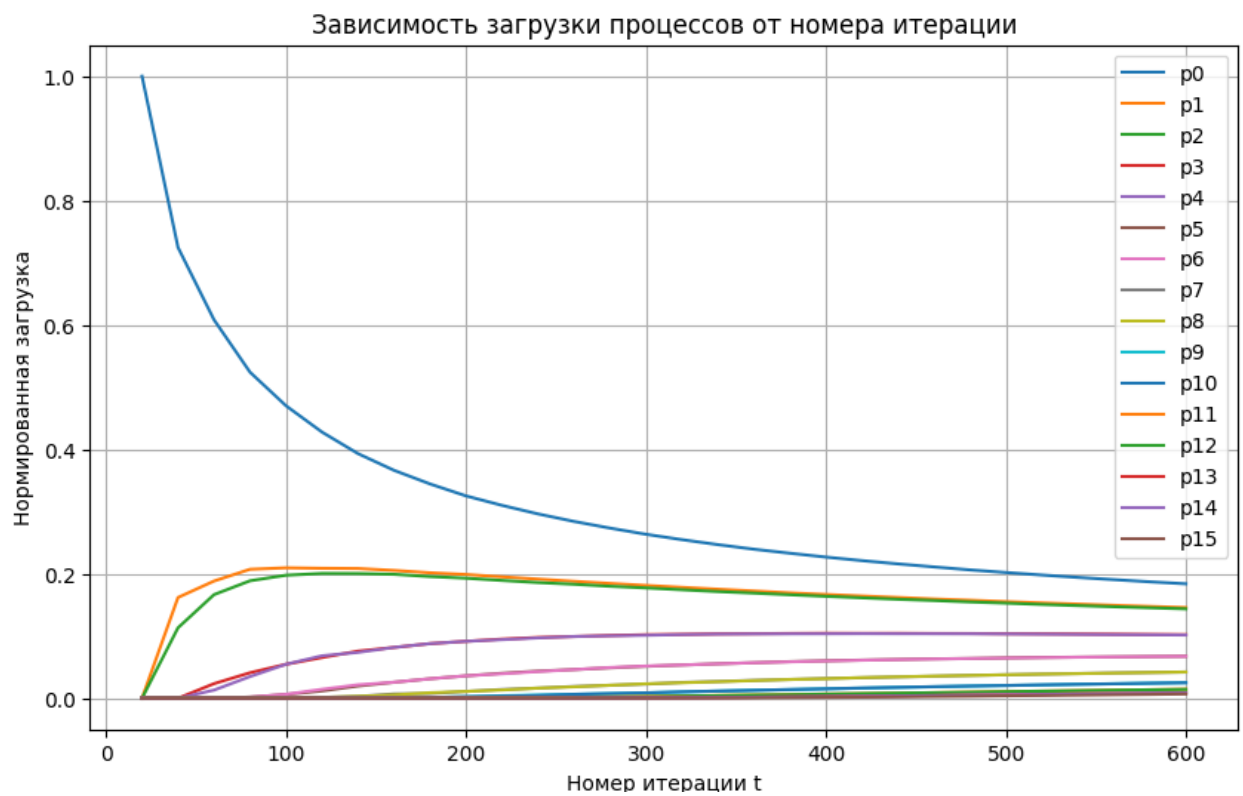
2 Графики

Для построения графиков были выбраны начальные значения $m = 600$, $k = 20$. На маленьком числе процессов хорошо видно перераспределение нагрузки. При большом - ситуация несколько хуже ввиду небольшого числа обменов.

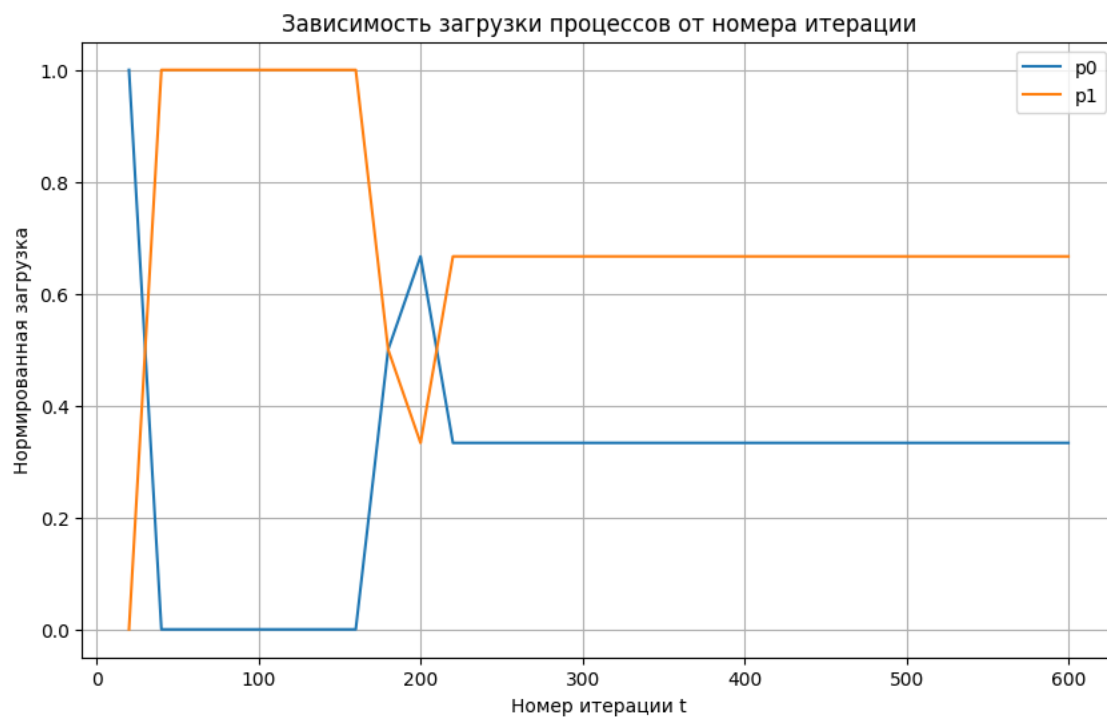
$$\bullet L_1 = \begin{cases} a \rightarrow ab, \\ b \rightarrow bc, \\ \omega = a; \end{cases}$$

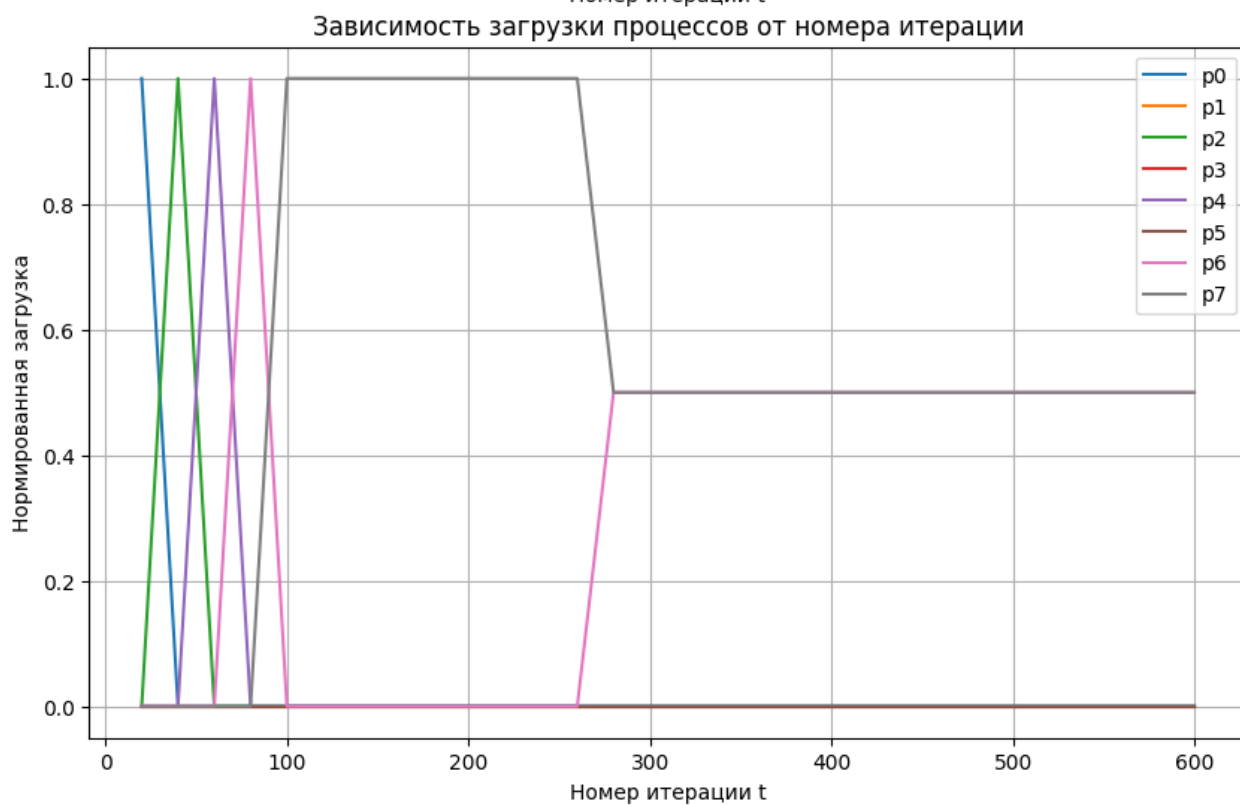
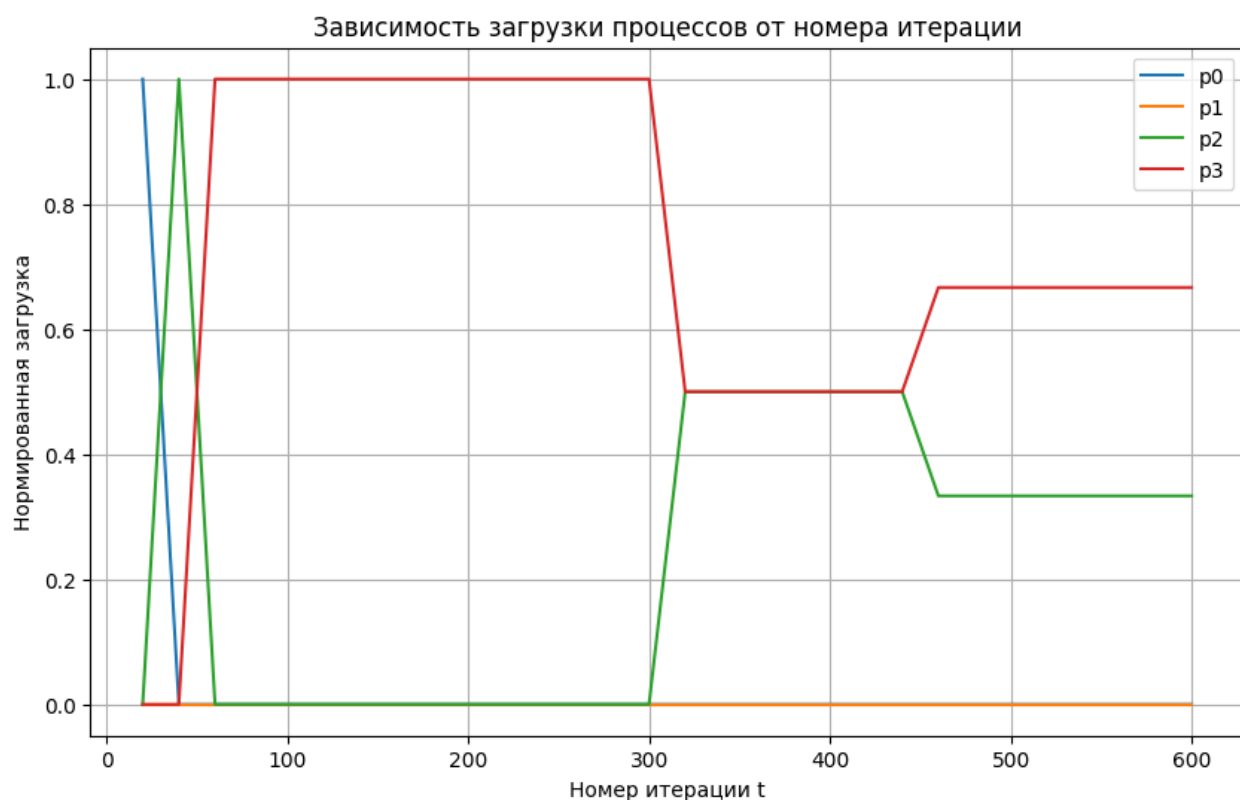


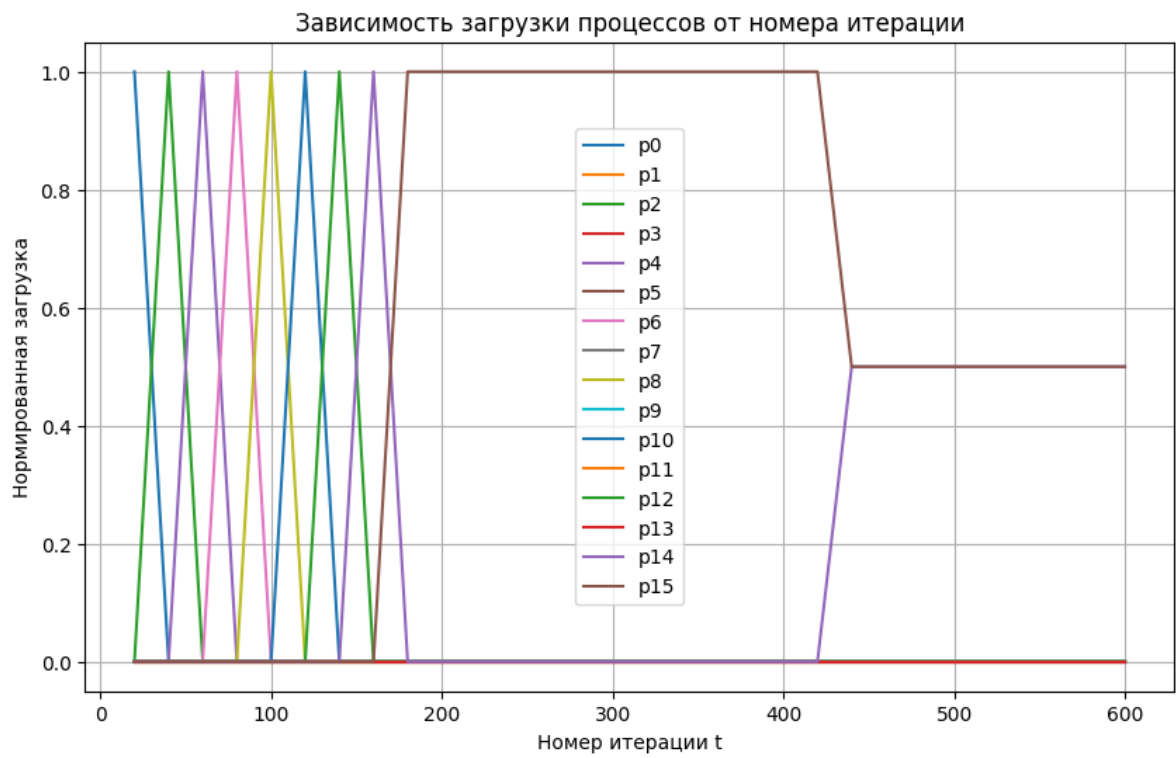




$$\bullet L_2 = \begin{cases} a \rightarrow aa [0.001], \\ \omega = a; \end{cases}$$







• $L_3 = \begin{cases} a \rightarrow ab [0.01], \\ b \rightarrow a [0.01], \\ \omega = a. \end{cases}$





